Code Instruction: Job Submission Behavior & Execution Schedules & Queue Modeling

Q1: Waiting Time Distribution (Start - Submit)

Dataset Used

- `master_2025_joblevel_terminal.csv.gz`

Code and Methodology

This section analyzes job waiting time using the terminal-level dataset. Jobs that remained in `PENDING` status or cancelled jobs without valid start timestamps were removed. Waiting time was computed as:

$$\text{Wait Time} = \text{Start} - \text{Submit}.$$

The code converts waiting time into hours and visualizes the distribution using histograms.

Main Findings

The waiting time distribution is highly right-skewed. Most jobs start quickly, while a smaller subset experiences extremely long queue delays.

Interpretation

The cluster behavior is dominated by tail events rather than typical jobs. Extreme delays motivate later analyses on workload heterogeneity and congestion.

Q2: Submission Patterns by Weekday and Hour of Day

Dataset Used

- `master_2025_joblevel_submission.csv.gz`

Code and Methodology

The code extracts weekday and hour information from submission timestamps and aggregates submission counts over time. Bar plots are used to visualize temporal submission behavior.

Main Findings

Job submissions are relatively stable across weekdays but vary strongly by hour of day. Submission activity peaks during afternoon and late evening hours.

Interpretation

The submission pattern reflects researcher and student activity schedules. Temporal bursts later help explain queue congestion patterns.

Q3: Runtime Distribution (End - Start)

Dataset Used

- `master_2025_joblevel_terminal.csv.gz`

Code and Methodology

Only completed jobs are included in this analysis. Runtime is computed as:

$$\text{Run Time} = \text{End} - \text{Start}.$$

The runtime distribution is visualized using histograms and descriptive statistics.

Main Findings

Runtime also exhibits strong right-skewness. Most jobs complete quickly, while a smaller number of jobs consume substantial execution time.

Interpretation

Long-running jobs may lock cluster resources and contribute disproportionately to resource contention.

Q4: Resource Request Distributions

Dataset Used

- `master_2025_joblevel_submission.csv.gz`

Code and Methodology

The code analyzes the distributions of:

- Requested CPUs
- Requested memory
- Requested nodes

Memory requests are converted from MB to GB for interpretability.

Main Findings

Most jobs request relatively small resources, but a small number of jobs request extremely large CPU or memory allocations.

Interpretation

The cluster workload is highly heterogeneous, with heavy-tail resource demand behavior.

Q5: GPU Request Distribution

Dataset Used

- `master_2025_joblevel_submission.csv.gz`
- `master_2025_joblevel_terminal.csv.gz`

Code and Methodology

GPU counts are extracted from the `ReqTRES` column using regular expressions. The code analyzes:

- GPU request distributions
- Waiting time distributions for GPU jobs

Main Findings

Most GPU jobs request only a small number of GPUs, but GPU jobs generally experience longer waiting times than CPU-only jobs.

Interpretation

GPU resources appear to be substantially more constrained than CPU resources, contributing to queue pressure.

Q6: CPU vs GPU Waiting Time Comparison

Dataset Used

- Combined submission and terminal datasets

Code and Methodology

Jobs are divided into GPU jobs and CPU-only jobs according to `GPU_Count`. Since waiting times are heavily skewed, the Mann–Whitney U test is used to compare waiting-time distributions.

Main Findings

GPU jobs exhibit significantly different waiting-time behavior (distributions of waiting times of CPU-only jobs and GPU jobs are different at a significance level of 5%) and generally wait longer than CPU-only jobs.

Interpretation

GPU queue congestion is statistically significant and reflects limited accelerator availability.

Q7: Predicting GPU Job Waiting Time

Dataset Used

- GPU jobs extracted from submission and terminal datasets

Code and Methodology

The notebook trains Random Forest and XGBoost regression models to predict GPU waiting time. Features include:

- Submission weekday and hour
- GPU count
- Requested CPUs
- Requested memory
- QOS
- Priority

Missing values are imputed, categorical variables are one-hot encoded, and numerical features are standardized.

Main Findings

The Random Forest model achieves better predictive performance than XGBoost for GPU waiting-time prediction.

Interpretation

Submission-time information contains meaningful signals for predicting future queue delays.

Q8: Workload Variable Construction (K-Means Clustering)

Dataset Used

- `master_2025_joblevel_submission.csv.gz`

Code and Methodology

The code constructs a workload variable using:

- Requested CPUs
- GPU count
- Requested memory

A log transformation is applied to reduce skewness, followed by standardization and K-means clustering.

Three workload categories are constructed:

- Light
- Medium
- Heavy

Main Findings

Heavy workloads are GPU-intensive and memory-intensive, while Light workloads correspond to smaller CPU-based jobs.

Interpretation

The workload variable provides a compact representation of overall job resource intensity.

Q9: CPU-only Job Waiting Time Prediction

Dataset Used

- CPU-only jobs extracted from submission and terminal datasets

Code and Methodology

A Random Forest regression model is trained to predict waiting time for CPU-only jobs. Similar preprocessing pipelines are applied as in Q7.

Main Findings

CPU-only waiting-time is harder to predict than GPU waiting-time.

Interpretation

We need more efficient features, and the distribution of CPU-only jobs' waiting times is more complicated.

Q10: Workload vs Wait Time Analysis

Dataset Used

- Workload categories constructed in Q8
- Waiting time data from terminal records

Code and Methodology

The code compares waiting-time distributions across workload categories using:

- Distribution plots
- Cumulative distribution functions (CDFs)

Main Findings

Heavy jobs exhibit substantially stronger right-tail waiting-time behavior.

Interpretation

Resource contention is concentrated among heavy jobs rather than uniformly distributed across all jobs.

Q11: Workload vs Run Time

Dataset Used

- Workload categories
- Runtime data from completed jobs

Code and Methodology
The notebook compares median and mean runtime across workload categories.
Main Findings
Heavy workloads exhibit significantly longer runtimes than Light and Medium workloads.
Interpretation
Long-running heavy jobs reduce system availability and intensify queue competition.

Q12: Wait Time vs Submission Time

Dataset Used

- Submission timestamps
- Waiting time data

Code and Methodology
The notebook analyzes average and median waiting times across different submission hours and weekdays.
Main Findings
Mean waiting time exhibits clear temporal spikes during specific hours.
Interpretation
Queue congestion is driven by burst-period overload and extreme delay events.

Q13: Wait Time vs Run Time

Dataset Used

- Runtime and waiting-time data from completed jobs

Code and Methodology
The code compares runtime and waiting time using visualization and correlation analysis.
Main Findings
Longer-running jobs tend to experience longer waiting times, although the relationship is not perfectly linear.
Interpretation
Queue delays arise from the interaction between runtime, workload intensity, and system-level congestion rather than runtime alone.